

SISTEMES DISTRIBUÏTS

Task2:Distributed Architecture

curs 2025-26

Estudiants: Alejandro Fernández Tules
Salma Jadiani Mahmoud

ÍNDICE

ÍNDICE	2
1.INTRODUCCIÓN	3
2.ARQUITECTURA DEL SISTEMA	4
2.1 COMPONENTES	4
2.2 DIAGRAMA DE ARQUITECTURA	4
2.3 COMUNICACIÓN	5
3. MODELO DE DATOS	6
3.1 ENTRADAS NUMERADAS	6
3.2 ENTRADAS NO NUMERADAS	6
3.3 IDEMPOTENCIA	6
4. WORKERS	7
4.1 WORKERS CON AWS LAMBDA	7
4.2 BATCH	7
4.3 DELAY ARTIFICIAL	7
5. ESCALADO DINÁMICO	8
5.1 MODELO DE ESCALADO	8
5.2 ADAPTACIÓN A LAMBDA AWS	8
5.3 RESULTADOS DEL ESCALADO	9
6.DESPLIEGUE	10
6.1 AWS	10
6.2 WORKLOADS EJECUTADOS	11
6.3 MÉTRICAS RECOGIDAS	11
7.RESULTADOS	12
7.1 ELASTIC WORKLOAD	12
7.2 STRESS TEST	12
7.3 ANÁLISIS DE LATENCIA	13
7.4 ANÁLISIS DEL THROUGHPUT	14
7.5 ANÁLISIS DEL ESTRÉS	14
7.6 HOTSPOT WORKLOAD	15
7.7 TABLA RESULTADOS	16
8.TOLERANCIA A FALLOS Y FIABILIDAD	17
9. DISCUSIÓN	18
10. CONCLUSIONES	19

1.INTRODUCCIÓN

Este proyecto implementa un sistema distribuido de compra de entradas capaz de procesar peticiones de entradas numeradas y no numeradas de forma asíncrona. El objetivo principal es garantizar la corrección del sistema en situaciones de concurrencia, evitando la sobreventa de entradas cuando se procesan varias peticiones al mismo tiempo.

El sistema se apoya en RabbitMQ como broker de mensajes, AWS Lambda como capa de workers stateless y PostgreSQL como persistencia. También se ha incluido escalado dinámico basado en la cola de trabajo, idempotencia a través de un identificador de solicitud, reintentos, una cola de mensajes rechazados, y métricas experimentales como el throughput, la latencia media, p50, p95, p99, y la tasa de éxito/fallo.

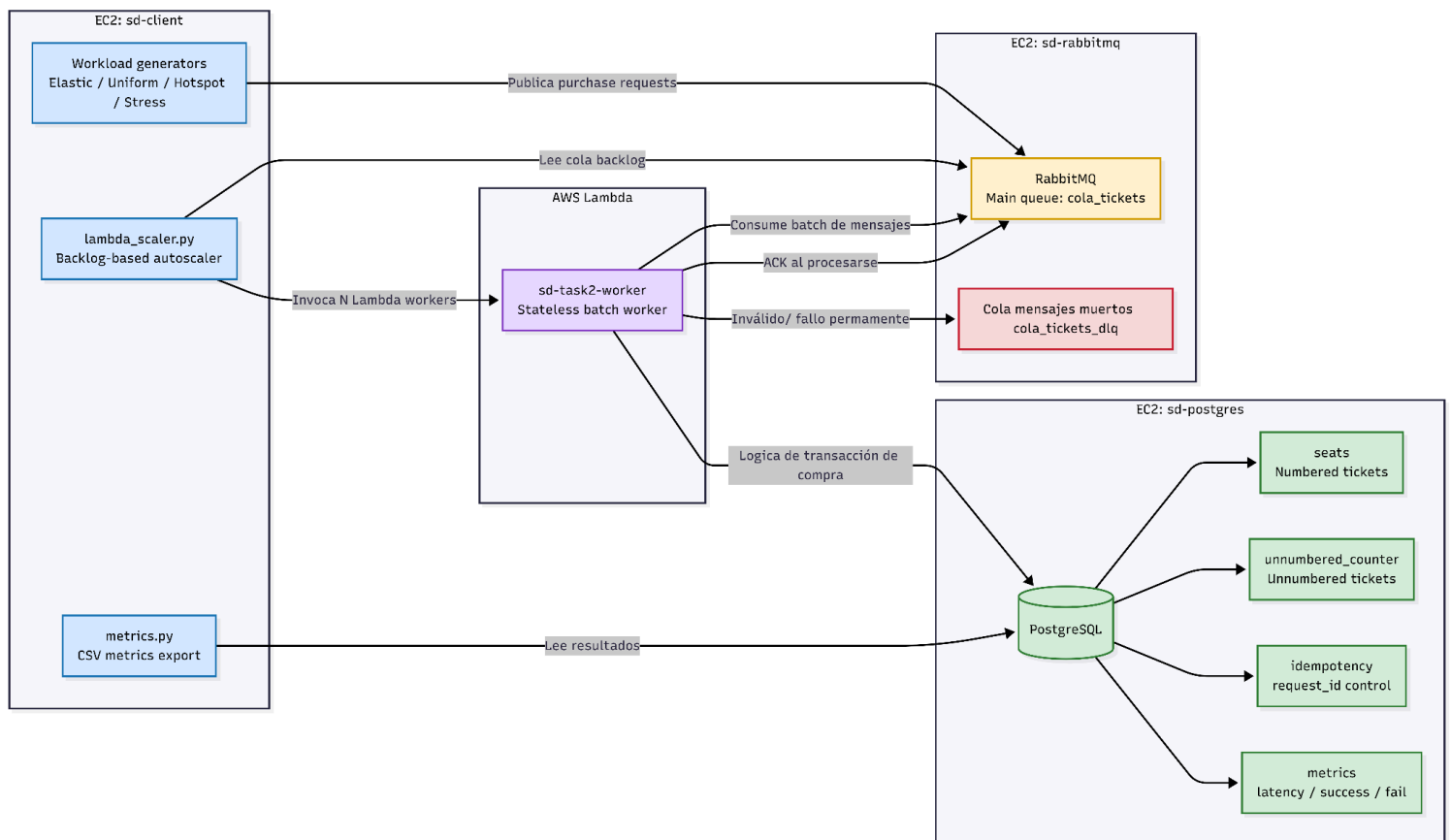
2.ARQUITECTURA DEL SISTEMA

2.1 COMPONENTES

El sistema está formado por los siguientes componentes:

- **Workload generator / Producer:** genera las peticiones de compra y las publica en RabbitMQ.
- **RabbitMQ:** broker de mensajes desplegado en una instancia EC2.
- **lambda_scaler.py:** componente encargado de monitorizar el backlog de RabbitMQ e invocar workers en AWS Lambda.
- **AWS Lambda workers:** funciones stateless que procesan lotes de peticiones de compra.
- **PostgreSQL:** base de datos persistente desplegada en una instancia EC2. Almacena los asientos, el contador de entradas no numeradas, la tabla de idempotencia y las métricas.

2.2 DIAGRAMA DE ARQUITECTURA



2.3 COMUNICACIÓN

El productor publica las peticiones de compra en RabbitMQ. El scaler consulta periódicamente el número de mensajes pendientes en la cola y calcula cuántas funciones Lambda debe invocar. Cada Lambda consume un lote de mensajes, procesa la compra dentro de PostgreSQL y guarda el resultado en la tabla de métricas. Una vez procesado correctamente el mensaje, se confirma mediante ACK en RabbitMQ.

3. MODELO DE DATOS

3.1 ENTRADAS NUMERADAS

En la tabla seats, cada entrada numerada corresponde a un asiento individual. Cada asiento solo puede venderse una vez. Cuando varias peticiones quieren comprar el mismo asiento al mismo tiempo, la decisión final la toma PostgreSQL con transacciones, para que sólo una petición pueda marcar el asiento como vendido.

3.2 ENTRADAS NO NUMERADAS

Las entradas no numeradas se controlan por medio de un contador global con un máximo de 100.000 entradas. El sistema eleva el contador únicamente cuando el número actual de entradas vendidas está por debajo del máximo permitido. Así se evita la sobreventa.

3.3 IDEMPOTENCIA

Cada petición incluye un identificador único request_id. El worker, antes de procesar una compra, comprueba que ese request_id no se haya procesado antes. Esto evita que una petición tenga un efecto doble en caso de reintentos o entregas duplicadas del mensaje.

3.4 Consistency y Race Conditions

La consistencia del sistema está garantizada delegando la decisión final de cada compra en PostgreSQL. Aunque varias funciones Lambda pueden procesar peticiones en paralelo, la operación crítica de venta se ejecuta dentro de una transacción en la base de datos. Si después otra petición intenta comprar el mismo asiento, la operación se rechaza porque el asiento aparece ya vendido.

Para las entradas no numeradas, la consistencia se mantiene mediante un contador global con un máximo de 100.000 entradas. El contador sólo se incrementa si todavía hay capacidad disponible. De esta manera, el sistema evita vender más entradas de las que hay disponible, incluso si hay mucha gente comprando al mismo tiempo.

4. WORKERS

4.1 WORKERS CON AWS LAMBDA

Los workers se implementan en funciones Lambda de AWS. Cada invocación procesa un número limitado de mensajes y después termina. Esto permite que los workers sean stateless, ya que no almacenan estado local entre ejecuciones. El estado persistente se mantiene en PostgreSQL y los mensajes pendientes permanecen en RabbitMQ.

4.2 BATCH

Cada invocación de una función Lambda procesa hasta `BATCH_SIZE` mensajes. En los experimentos realizados configuramos `BATCH_SIZE` a 10 mensajes.

4.3 DELAY ARTIFICIAL

Se introdujo un retardo artificial de 100 ms dentro de la lógica del worker para simular un procesamiento más realista. Esto permite observar mejor los efectos de la concurrencia, el backlog y el escalado dinámico durante las pruebas.

5. ESCALADO DINÁMICO

5.1 MODELO DE ESCALADO

El modelo general de escalado calcula el número de workers necesarios en función del backlog actual, la tasa de llegada de nuevas peticiones, el tiempo objetivo de respuesta y la capacidad de procesamiento de cada worker.

La fórmula general queda:

$$N = \text{ceil}((B + \lambda * Tr) / (C * Tr))$$

Donde:

N es el número de workers que se necesitan

B es el backlog real en RabbitMQ

λ es la tasa de llegada de nuevas peticiones

Tr es el tiempo objetivo de respuesta

C es la capacidad de procesamiento de un worker

La parte superior de la fórmula representa el trabajo total que debe procesarse: los mensajes que ya están esperando más los mensajes que llegarán durante el tiempo objetivo. La parte inferior muestra cuánto puede procesar un worker durante ese intervalo.

Por ejemplo, si RabbitMQ tiene un backlog de 500 mensajes pendientes y cada Lambda procesa un lote máximo de 10 mensajes, el scaler calcularía inicialmente $\text{ceil}(500 / 10) = 50$ invocaciones. Sin embargo, como el sistema está limitado a $N_{\text{max}} = 10$, solo se lanzan 10 invocaciones Lambda en ese ciclo. Esto permite adaptar el procesamiento al tamaño de la cola, pero respetando los límites del entorno de AWS Academy.

5.2 ADAPTACIÓN A LAMBDA AWS

En nuestra implementación, los workers son funciones AWS Lambda que no están ejecutándose todo el tiempo. Cada Lambda recibe un lote de mensajes y se cierra. Por esta razón el modelo se adaptó a invocaciones por lotes:

$$N = \min(N_{\text{max}}, \text{ceil}(B / \text{batch_size})) \text{ (redondeando arriba = ceil)}$$

Donde:

B es el backlog actual en RabbitMQ.

batch_size es el tamaño máximo de mensajes procesados por cada Lambda.

Nmax es el número máximo de Lambdas que puede invocar el scaler por ciclo.

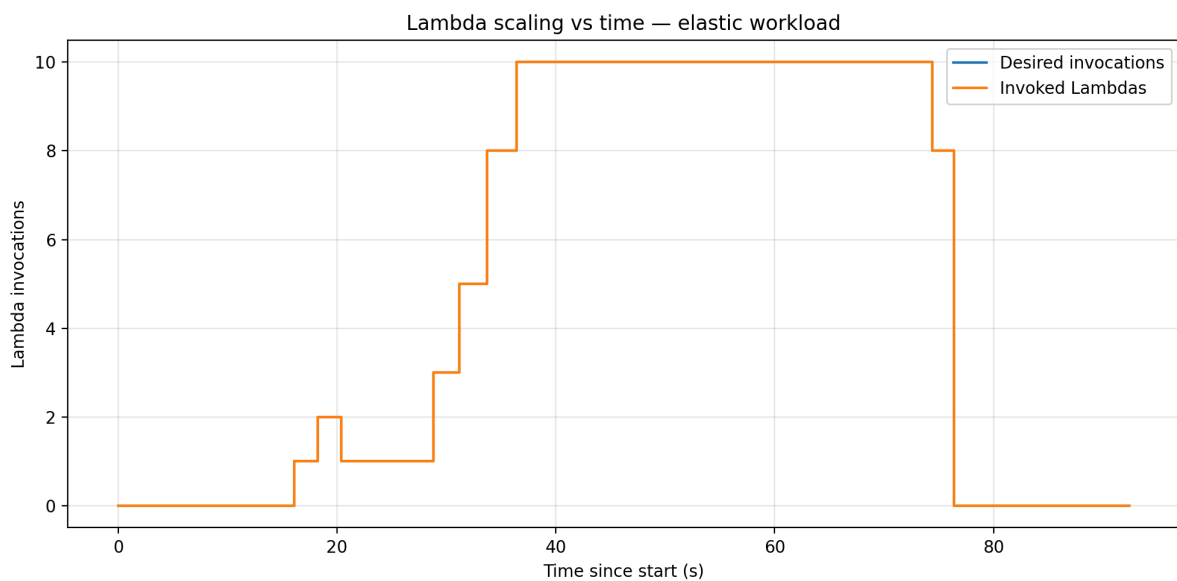
En experimentos:

batch_size=10

nmax = 10

Se usó el límite Nmax = 10 por las limitaciones del entorno AWS Academy.

5.3 RESULTADOS DEL ESCALADO



Estas figuras muestran el comportamiento del scaler ante el incremento del backlog. A medida que la carga se incrementa, el número de invocaciones Lambda también lo hace hasta llegar al máximo configurado. A medida que el backlog se va reduciendo, el número de invocaciones vuelve a disminuir.

6.DESPLIEGUE

6.1 AWS

Se hizo el despliegue en AWS Academy en la región us-east-1. Se utilizaron tres máquinas EC2:

- sd-client: generación de workloads, escalador en ejecución y extracción de métricas.
- sd-rabbitmq: broker RabbitMQ.
- sd-postgres : base de datos PostgreSQL.

La capa de procesamiento fue implementada mediante la función Lambda de AWS sd-task2-worker.

6.2 WORKLOADS EJECUTADOS

Se ejecutaron los siguientes tipos de carga:

- **Elastic workload:** carga variable con fases de baja carga, subida progresiva, pico, carga alta sostenida y enfriamiento.
- **Uniform workload:** peticiones distribuidas de forma uniforme entre los asientos.
- **Hotspot workload:** el 80% de las peticiones se concentran sobre el 5% de los asientos.
- **Stress workload:** ráfagas grandes de peticiones para analizar el límite de capacidad del sistema.

6.3 MÉTRICAS RECOGIDAS

El sistema recoge las siguientes métricas:

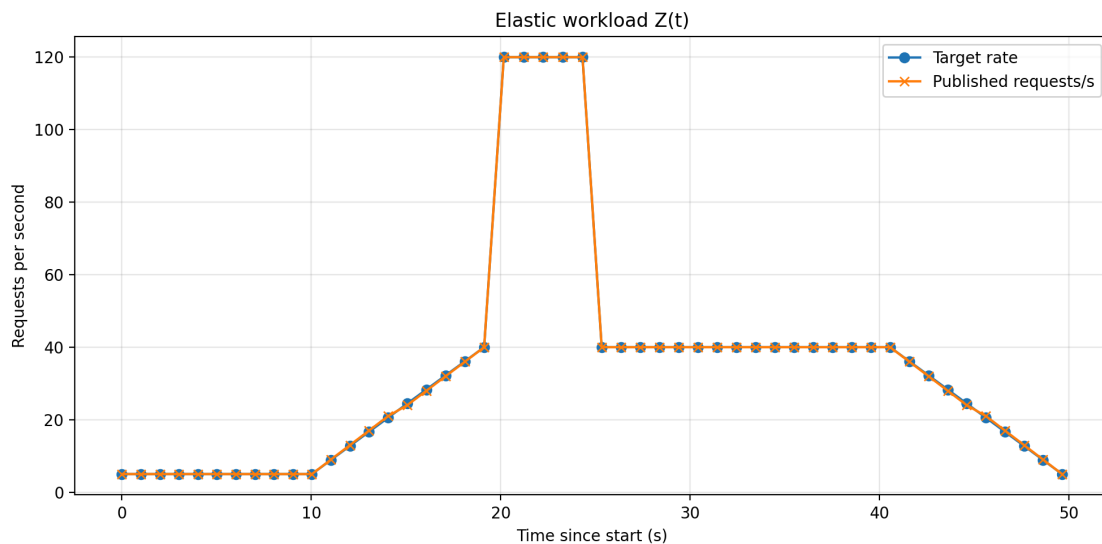
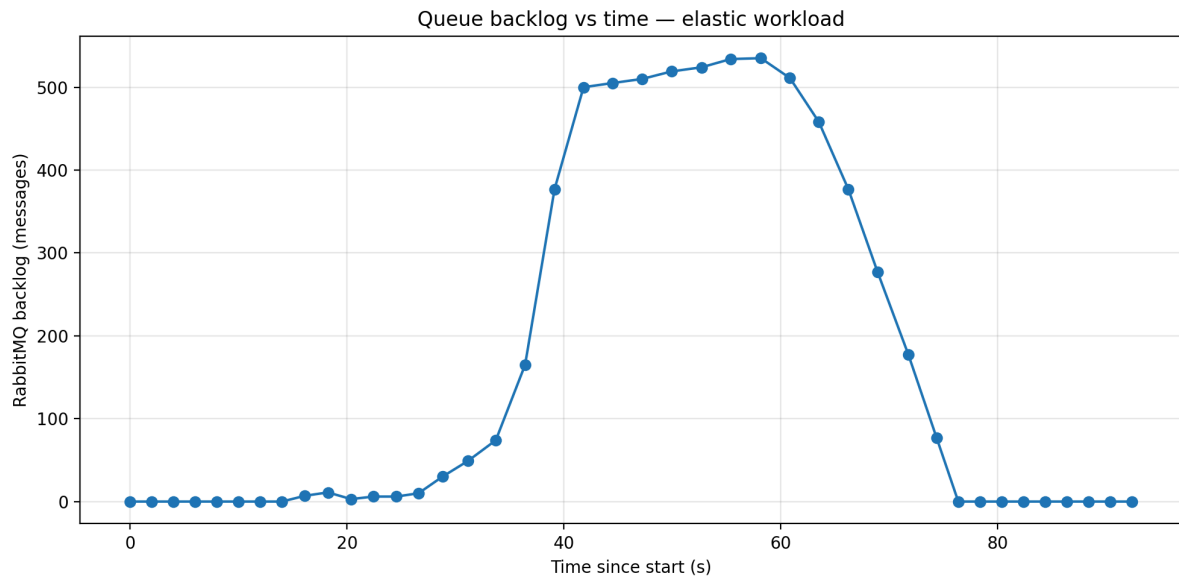
- Número total de operaciones.
- Compras exitosas.
- Compras fallidas.
- Peticiones duplicadas.
- Throughput.
- Latencia media.
- Latencia p50.
- Latencia p95.

- Latencia p99.
- Backlog de RabbitMQ.
- Número de invocaciones Lambda.

7.RESULTADOS

7.1 ELASTIC WORKLOAD

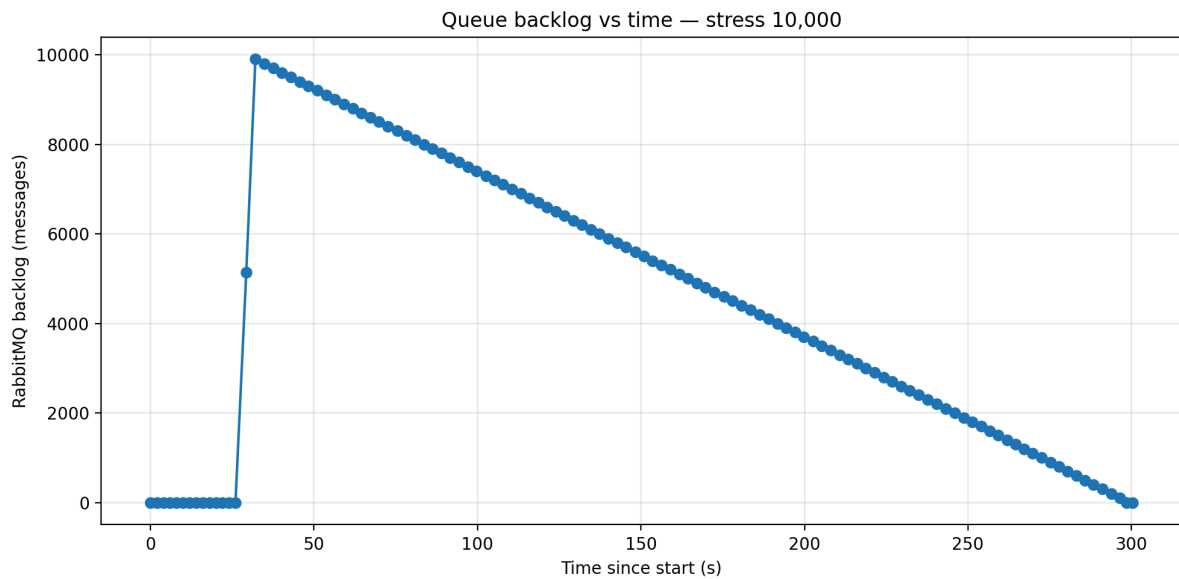
La carga elástica permite comprobar que el sistema reacciona ante los cambios de carga. En las fases de pico y carga alta sostenida se observa un crecimiento del backlog de RabbitMQ. Luego, cuando el scaler vuelve a invocar Lambdas y la carga baja, la cola se va vaciando de forma progresiva.



7.2 STRESS TEST

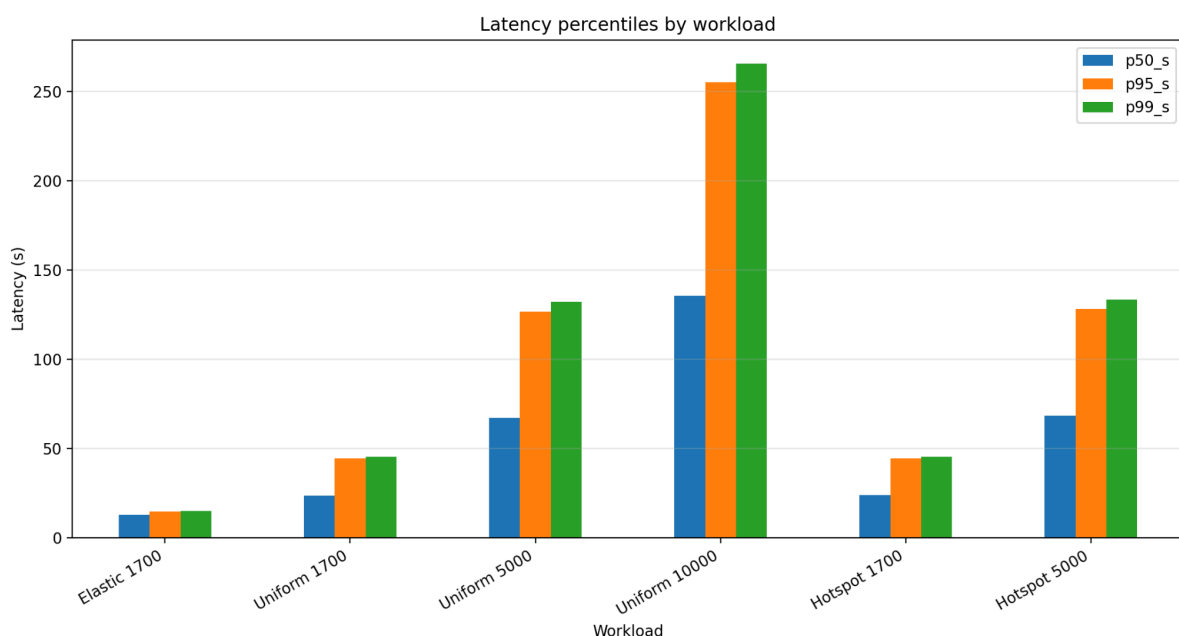
El stress test de 10.000 peticiones muestra cómo se comporta el sistema cuando le llegan muchos mensajes a la vez. La cola se llena rápidamente con

casi 10.000 mensajes y, posteriormente, se vacía de forma progresiva a medida que las Lambdas procesan los lotes.



Este comportamiento muestra que RabbitMQ permite absorber picos de carga, pero la latencia crece cuando el número de mensajes pendientes es elevado.

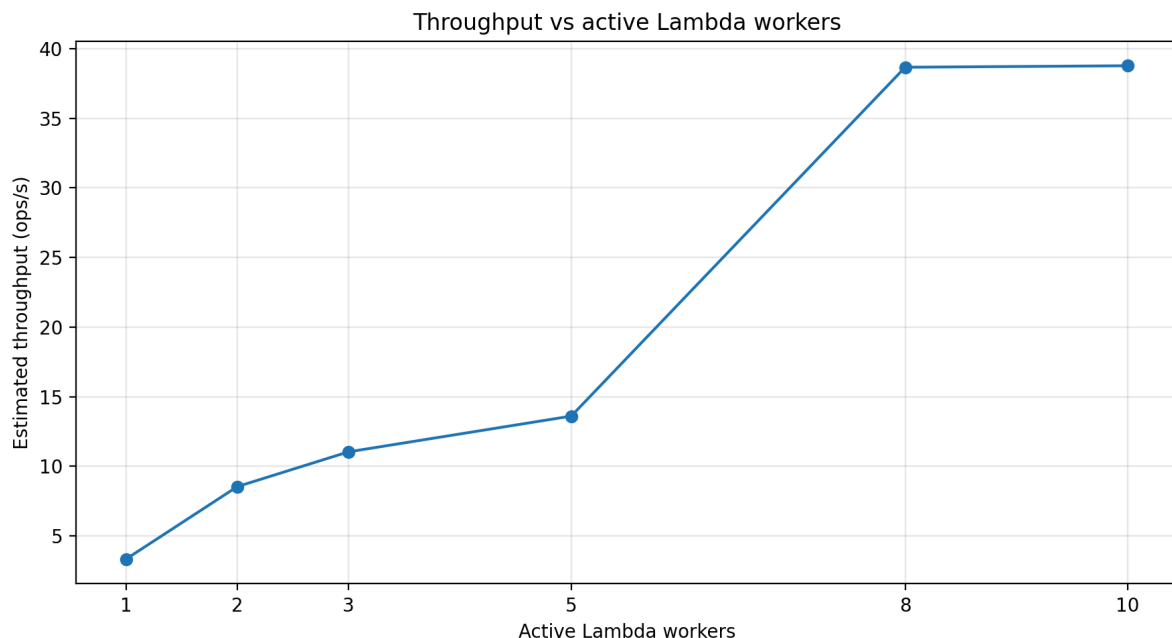
7.3 ANÁLISIS DE LATENCIA



Los resultados demuestran que la latencia aumenta al aumentar el tamaño del workload. En cargas pequeñas, las peticiones esperan poco tiempo en cola. En cambio, en cargas de trabajo grandes como el de 10.000 peticiones,

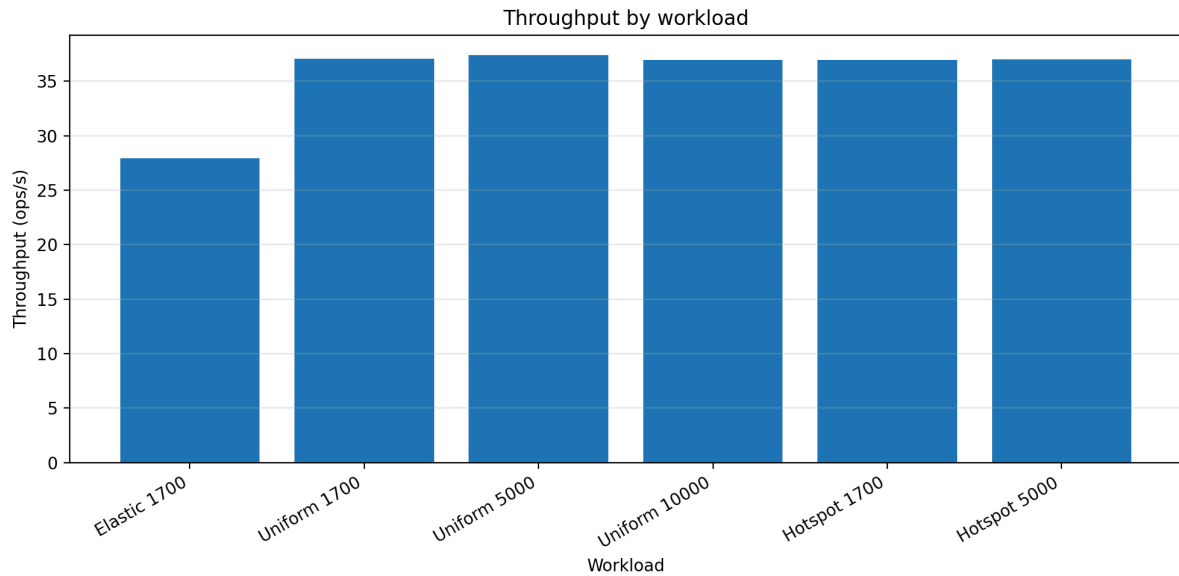
muchas solicitudes deben esperar a que las Lambdas procesen los mensajes anteriores, lo que aumenta la latencia p50, p95 y p99.

7.4 ANÁLISIS DEL THROUGHPUT



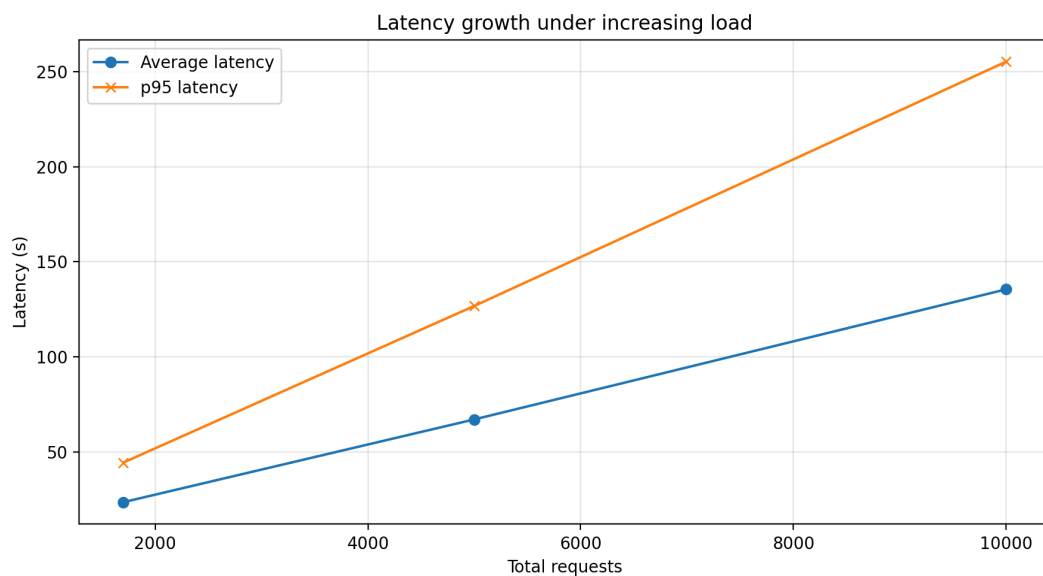
A continuación se muestra la gráfica del throughput estimado en función del número de workers Lambda activos durante el experimento de elastic scaling. Se ha utilizado el número de invocaciones Lambda registradas por el scaler y el procesamiento efectivo observado en cada intervalo temporal para obtener esta relación.

Se puede observar que el throughput va en aumento a medida que se incrementa la cantidad de workers, hasta estabilizarse en torno a las 38 operaciones por segundo cuando el sistema llega a tener entre 8 y 10 workers activos. Esto significa que, a partir de ese punto, el sistema se acerca a su capacidad máxima de procesamiento dentro de los límites configurados en AWS Academy.



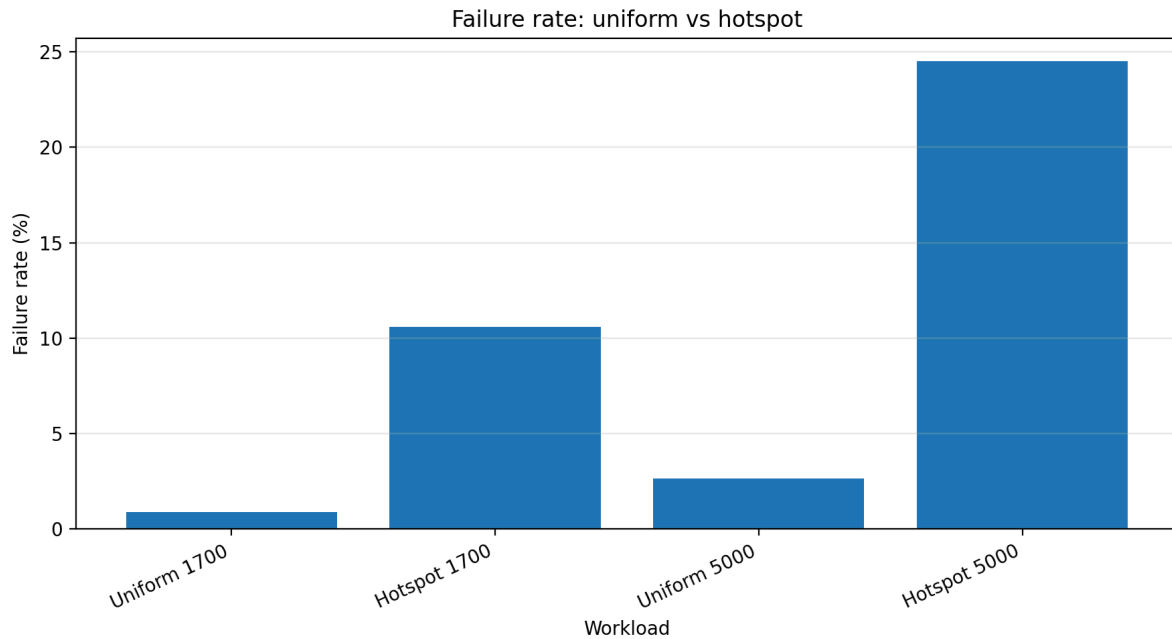
El throughput se mantiene en los workloads grandes en alrededor de 37 operaciones por segundo. Esto implica que el sistema alcanza un procesamiento estable, limitado por el número máximo de invocaciones Lambda, el tamaño del lote y el retardo artificial de 100 ms.

7.5 ANÁLISIS DEL ESTRÉS



Cuando el número de peticiones pasa de 1.700 a 5.000 y 10.000, el throughput se mantiene prácticamente estable, pero la latencia crece de forma significativa. Esto quiere decir que el sistema está saturado: sigue procesando a un ritmo similar, pero las peticiones pasan más tiempo esperando en RabbitMQ.

7.6 HOTSPOT WORKLOAD



El workload del hotspot produce una tasa de fallos mucho mayor que el workload uniforme. Esto se debe a que muchas peticiones intentan comprar el mismo pequeño conjunto de asientos. Sin embargo, el sistema conserva la corrección, ya que las compras conflictivas se rechazan en lugar de generar sobreventa.

7.7 TABLA RESULTADOS

Workload	Total ops	Success	Fail	Duplicados	Throughput (ops/s)	Latencia media (s)	p50 (s)	p95 (s)	p99 (s)
Elastic 1700	1700	1684	16	0	27.93	9.71	12.70	14.51	14.80
Uniform 1700	1700	1685	15	0	37.07	23.57	23.56	44.40	45.20
Uniform 5000	5000	4869	131	0	37.38	67.09	67.05	126.78	132.14
Uniform 10000	10000	9515	485	0	36.98	135.48	135.43	255.22	265.75
Hotspot 1700	1700	1520	180	0	36.97	23.76	23.77	44.40	45.26
Hotspot 5000	5000	3775	1225	0	37.04	68.35	68.46	128.09	133.39

La tabla muestra que el throughput se mantiene estable alrededor de 37 operaciones por segundo en los workloads grandes. En cambio, sí que se crece de forma clara la latencia, sobre todo en el workload uniforme de 10.000 peticiones donde el p95 supera los 255 segundos. Esto indica que cuando la carga es mayor que la capacidad de procesamiento disponible, las peticiones se acumulan en RabbitMQ.

También se ve que el workload hotspot genera más tasa de fallos que el workload uniforme. Esto no quiere decir que el sistema falle técnicamente, sino que muchas peticiones compiten por los mismos asientos y son rechazadas correctamente cuando esos asientos ya han sido vendidos. Por lo tanto, el aumento de los errores de hotspot confirma la existencia de una mayor contención, pero también demuestra que el sistema evita la sobreventa.

8.TOLERANCIA A FALLOS Y FIABILIDAD

El sistema ha sido diseñado considerando que en una arquitectura distribuida basada en colas, pueden surgir fallos durante el procesamiento de los mensajes. RabbitMQ trabaja con la semántica de **at-least-once** delivery, lo

que implica que un mensaje puede ser entregado más de una vez si un worker falla antes de confirmar que lo ha procesado con un ACK. Para que esta posible repetición no dé lugar a ventas duplicadas, cada petición incluye un identificador único `request_id`, que se emplea como mecanismo de idempotencia.

El worker, al recibir una petición, primero comprueba que ese `request_id` no se haya procesado anteriormente. Si se trata de una petición nueva, la lógica de compra se ejecuta dentro de una transacción de PostgreSQL. Si por el contrario, el identificador ya existe, la petición se marca como duplicada y no se vuelve a aplicar la operación sobre la base de datos. De este modo, si RabbitMQ entrega dos veces el mismo mensaje, el sistema se asegura de que una misma compra solo se ejecute una vez.

Además, el sistema cuenta con un mecanismo de reintentos para gestionar errores temporales, como fallos puntuales de conexión con la base de datos o errores momentáneos en la ejecución del worker. Si un mensaje no puede ser procesado correctamente, entonces se reintenta hasta un número máximo de intentos. Si el mensaje continúa fallando o es inválido, se envía a una dead-letter queue o cola de mensajes fallidos. De este modo se evita que un mensaje mal formado bloquee la cola principal y que el resto de peticiones no puedan continuar procesándose.

Se realizaron varias pruebas específicas para validar la fiabilidad del sistema. Primero se comprobó la idempotencia enviando dos veces una petición con el mismo `request_id`. El sistema procesó bien la primera petición y detectó la segunda como duplicada, asegurándose de que el asiento se vendiera sólo una vez. En segundo lugar se comprobó el límite de entradas no numeradas haciendo un test de frontera, inicializando el contador de entradas vendidas a 99.999 y enviando dos nuevas peticiones de compra. El sistema aceptó una y rechazó la otra, con lo que el número final quedó en 100.000. La prueba confirma que el tope máximo se cumple de manera correcta y que no se genera sobreventa.

9. DISCUSIÓN

La arquitectura separa de forma correcta la recepción de peticiones, el procesamiento y la persistencia. RabbitMQ sirve para que los workers y el productor estén desacoplados y para absorber picos de carga. AWS Lambda

ofrece trabajadores sin estado que pueden llamarse de forma dinámica. PostgreSQL asegura la consistencia de las operaciones críticas a través de transacciones.

El principal inconveniente encontrado es que el throughput está limitado por el número máximo de llamadas Lambda, el tamaño del lote y el retardo artificial añadido en el procesamiento. Si la carga supera la capacidad de procesamiento, las peticiones se van acumulando en RabbitMQ y la latencia va subiendo.

El workload hotspot demuestra que la contención incrementa el número de compras fallidas, pero el sistema mantiene la corrección ya que las peticiones conflictivas son rechazadas correctamente.

9.1 PREGUNTAS CONCEPTUALES

Q1. Consistency vs Scalability Trade-off

Las peticiones se procesan de forma asíncrona con RabbitMQ y AWS Lambda, pero la venta de una entrada se decide finalmente en PostgreSQL con transacciones. Esto evita que se venda dos veces un mismo asiento o que los billetes sin numerar superen los 100.000.

Con esta decisión se da prioridad a la corrección frente a la máxima escalabilidad. Con una consistencia más débil el sistema podría ganar rendimiento y reducir la latencia, pero con más riesgo de sobreventa. Pero una consistencia más estricta podría aumentar los bloqueos y reducir el throughput. Por eso, el diseño tiene como objetivo equilibrar la escalabilidad del procesamiento y la consistencia fuerte de la base de datos.

Q2. Fault Tolerance vs Performance Trade-offs

Se logra tolerancia a fallos mediante la entrega al menos una vez, reintentos, idempotencia con request_id y una cola de mensajes muertos para los mensajes inválidos. Estos mecanismos mejoran la fiabilidad, pero suponen cierto coste adicional.

Si hay demasiados errores, el throughput efectivo podría bajar porque parte del sistema se pasará re-procesando mensajes. Aun así, este coste es aceptable, pues evita compras duplicadas, mejora la recuperación ante errores y deja la cola principal libre de mensajes problemáticos.

9.2 USO DE LA IA

En el transcurso de la práctica se emplearon herramientas de inteligencia artificial como apoyo puntual en ciertas fases del trabajo. Sobre todo los utilizaron para revisar la estructura de la documentación, mejorar la redacción de algunos apartados técnicos y ayudar a interpretar los resultados obtenidos en las pruebas.

También fueron empleados como soporte para plantear posibles comandos, revisar errores durante el despliegue y organizar los gráficos de resultados. Sin embargo, los miembros del grupo realizaron y revisaron la implementación, la configuración del sistema en AWS, la ejecución de los experimentos y la validación de los resultados.

Las herramientas de IA se usaron por tanto como apoyo auxiliar para mejorar la claridad del trabajo y resolver dudas concretas, pero las decisiones de diseño, las pruebas finales y el análisis del comportamiento del sistema se basan en el trabajo realizado durante la práctica.

10. CONCLUSIONES

El sistema desarrollado logra los objetivos fundamentales de la práctica. Procesa compras de entradas de forma asíncrona, utiliza workers stateless, mantiene el estado en una base de datos persistente y aplica escalado dinámico en función del backlog de RabbitMQ.

Los resultados experimentales muestran que el sistema funciona correctamente bajo diferentes workloads: uniformes, hotspot, elásticos y de estrés. Con cargas elevadas, el throughput se estabiliza alrededor de 37 operaciones por segundo, y la latencia se incrementa por la acumulación de mensajes en la cola.

El sistema, aun ante concurrencia y peticiones conflictivas, evita la sobreventa mediante transacciones en PostgreSQL, control de idempotencia y validaciones específicas para entradas numeradas y no numeradas.